

UNIVERSIDADE DE FORTALEZA - UNIFOR

Curso de Análise e Desenvolvimento de Sistemas

F105 – Datamining e Webscraping

**Monitoramento de Preços da Cesta Básica: Coleta de Dados via API
Interna do Supermercado São Luiz**

Trabalho de conclusão da disciplina

Integrantes:

Nelivaldo Moreira Lyra – Matrícula 2415633

Cauã de Oliveira Silva Paiva – Matrícula 2415484

Fortaleza – CE

Setembro de 2026

1. Tema e Pergunta de Negócio

Queríamos um projeto de web scraping para verificação de produtos da cesta básica, escolhemos 5 para comparação de preços: **arroz, feijão, açúcar, óleo e ovos**, em três redes de supermercado online: **Carrefour, Pão de Açúcar e São Luiz**.

Pergunta de negócio: quais marcas de cada item têm o menor e o maior preço, qual a diferença entre elas, e qual mercado é sistematicamente mais barato por item?

2. Fonte dos Dados

A fonte de dados escolhida foi a loja online do supermercado São Luiz, Carrefour e Pão de Açúcar. Os produtos ficaram restrita aos cinco itens que compõem a cesta básica considerada trabalho: arroz, feijão, açúcar, óleo e ovos (o item leite foi substituído por ovos após verificação de que a categoria de leite não estava disponível de equivalente numa fonte escolhida).

Links das fontes:

- Carrefour: (`mercado.carrefour.com.br`) / `GET /busca?term=...`
- Pão de Açúcar: (`paodeacucar.com`) / `POST api.vendas.gpa.digital/pa/search/search`
- São Luiz: (`mercadinhossaoluiz.com.br`, `market\_id=355`) / `GET merconnect.mercadapp.com.br/mapp/v3/markets/355/items`

3. Método de Coleta

Cada loja exigiu uma estratégia diferente, sempre começando por inspecionar o tráfego de rede (DevTools → Network → Fetch/XHR).

Carrefour — scraping de HTML. A busca devolve HTML já renderizado no servidor. Usa-se requests + BeautifulSoup, extraíndo cada card via `a[data-testid="search-product-card"]`, com nome (`h2`), preço atual (`span.text-price-default`) e preço original (`span.line-through`).

Pão de Açúcar — API interna. O HTML inicial não contém produtos (hidratados via JS). A chamada que popula a grade é `POST .../pa/search/search`, com payload contendo `terms`, `page`, `sortBy`, `storeId=461` e um `userHash` gerado pelo frontend. A resposta é JSON com `totalPages` e `products[]` já estruturados. Dispensa Selenium/Playwright. O ponto frágil é o `userHash`, com validade curta.

São Luiz — API com token Bearer. O site é SPA React; o HTML é só `<div id="root">`. A API do `mercadapp` exige `Authorization: Bearer <token>`, com vida curta. Um script separado (`token/get_token_sao_luiz.py`) abre o site via Playwright, intercepta a requisição e grava o token em `.env`, lido automaticamente pelo raspador.

Limite de coleta. Cada raspador pega no máximo **5 produtos por item básico** — suficiente para a análise e muito abaixo do catálogo completo. Os três escrevem no mesmo dados/dados_brutos.csv via append, com 9 colunas fixas: item_basico, nome_produto, preco, preco_original, desconto_percentual, codigo_produto, categoria_busca, fonte, url_produto.

Pipeline automatizado. Um único python setup.py executa em ordem: limpar CSV → capturar token → coletar São Luiz → Carrefour → Pão de Açúcar → tratar → analisar. Se uma etapa falha, o pipeline para.

Justificativa da escolha (API e seletores): usamos API quando disponíveis para pegar os dados já estruturados em JSON (nome, preço, preço original, código de barras, estoque, categoria), mas usamos seletores no Carrefour por não ter encontrado uma API disponível. O impacto pode acontecer caso haja alguma mudança no layout da página podendo precisar de atualizações, as APIs usadas para alimentar os 2 sites não é oficialmente pública mas foram conseguidas por meio de testes no Postman e verificações manuais no DevTools, o projeto só está sendo usado a nível educacional usando uma técnica legítima e amplamente usada em projetos de coleta de dados, mas que traz implicações importantes discutidas na seção de Aspectos Éticos e Legais.

Autenticação e paginação: a API do Mercado São Luis exige um cabeçalho Authorization: Bearer <token>. Esse token é gerado do lado do cliente (navegador) e tem vida curta — na prática, verificamos que um token deixa de ser aceito (erro HTTP 401) após alguns minutos, exigindo nova captura via DevTools a cada execução manual do script, por isso, o setup faz questão de gerar ele. A paginação pode ser feita de forma simples, incrementando o parâmetro page e consultando o campo has_next_page da resposta para saber quando parar.

Limite de página por categoria: por padrão, a API pode devolver centenas de itens em algumas categorias (ex.: mais de 150 produtos na categoria de óleos). Como a atividade exige um mínimo de 30 registros no total (e não por item), o script foi ajustado para coletar no máximo 5 itens por tipo e em cada fonte — o suficiente para ultrapassar o mínimo exigido somando as cinco categorias, sem gerar volume de requisições desnecessário - no total pode ser gerado até 75 itens, 25 por fonte.

Incidente observado durante os testes: ao tentar coletar um número maior de páginas em sequência rápida, a API retornou erro HTTP 403 (Forbidden), inclusive em endpoints não relacionados à coleta (como um endpoint de avisos do próprio site), sugerindo um bloqueio temporário por parte de um mecanismo de proteção contra abuso (rate limiting / WAF) associado ao IP de origem das requisições. Esse episódio reforçou, na prática, a necessidade dos cuidados discutidos na seção seguinte.

4. Pré-processamento

Tratamento de valores ausentes e duplicados: Registros sem nome_produto ou preco foram descartados (campos essenciais para a análise); quando preco_original vinha vazio, foi preenchido com o próprio preco (significa "sem desconto ativo", não um dado faltante de fato). Duplicatas foram removidas por codigo_produto (chave mais confiável que o nome do produto).

Padronização: Preços foram convertidos para float com 2 casas decimais de forma tolerante a formato (aceita tanto o número já numérico salvo pelo coleta.py quanto uma eventual string em formato BR, ex. "R\$ 1.234,56"). Nomes de produto foram normalizados (espaços extras removidos).

Filtro de relevância: Mantém só produtos cujo nome ****começa**** com o termo do item: `^ARROZ`, `^FEIJAO`, `^ACUCAR`, `^OLEO DE (SOJA|MILHO|CANOLA|GIRASSOL|COCO|...)`, `^OVOS?`. Isso é necessário porque no São Luiz a categoria "Temperos" (13799) mistura óleos com molhos e maionese, e "Mercearia" (13797) mistura ovos com sopas e mistura para bolo. O filtro também remove falsos positivos como `Coca-Cola Sem Açúcar`, `Adoçante`, `Tempero Sazón` e `Sopa Maggi` no Carrefour.

Os itens removidos por esse filtro foram salvos separadamente em `itens_excluidos.csv`, preservando rastreabilidade da limpeza para fins de auditoria/relatório.

Script: `tratamento.py`, que lê `dados_brutos.csv` e gera `dados_tratados.csv`.

5. Mineração de Dados, Análises e Visualizações

Script `analise.py`, que gera cinco CSVs em `dados/`, quatro gráficos em `graficos/`, um dashboard e `insights.txt`.

Estatísticas descritivas (calculadas em `analise.py`, salvas em `estatisticas_descritivas.csv`):

Item	N	Média	Mediana	Mín	Máx	Desvio
Arroz	15	15,14	9,95	4,39	36,90	10,15
Feijão	15	10,08	8,94	5,98	21,99	4,12
Açúcar	11	4,96	3,99	2,69	8,99	2,03
Óleo	10	10,47	8,89	5,69	19,99	z5,05
Ovos	13	15,95	17,36	5,98	24,31	5,53

5.2 Preço médio por item × fonte

Item	Carrefour	Pão de Açúcar	São Luiz
Arroz	13,53	17,27	14,63
Feijão	8,65	9,61	11,99
Açúcar	4,02	5,55	4,92
Óleo	7,73	13,21	—
Ovos	10,47	18,77	20,38

O Carrefour é o mais barato em todos os itens. O Pão de Açúcar tende a ser o mais caro, com destaque para ovos (quase o dobro do Carrefour). O São Luiz fica em posição intermediária.

5.3 Gráficos

1. grafico1_preco_medio.png — preço médio por item (todas as fontes).
2. grafico2_distribuicao_precos.png — boxplot por item.
3. grafico3_menor_maior_preco.png — menor × maior preço por item.
4. grafico4_preco_medio_por_fonte.png — preço médio por item × mercado (responde à comparação entre fontes).
5. dashboard.png — combina os gráficos 1 e 3.

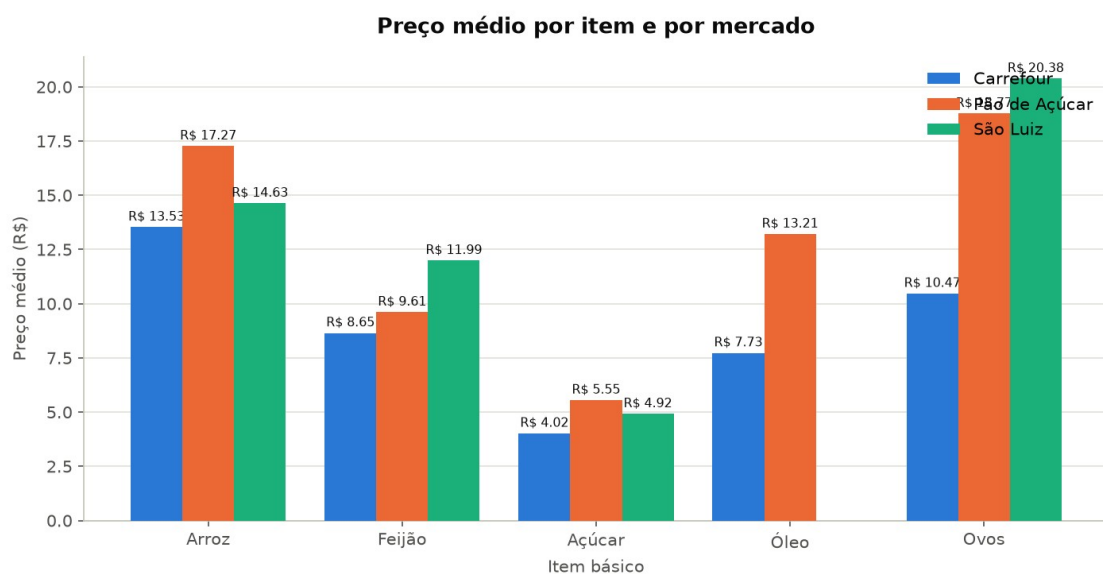


Figura 1 — Preço médio por item básico e fonte.

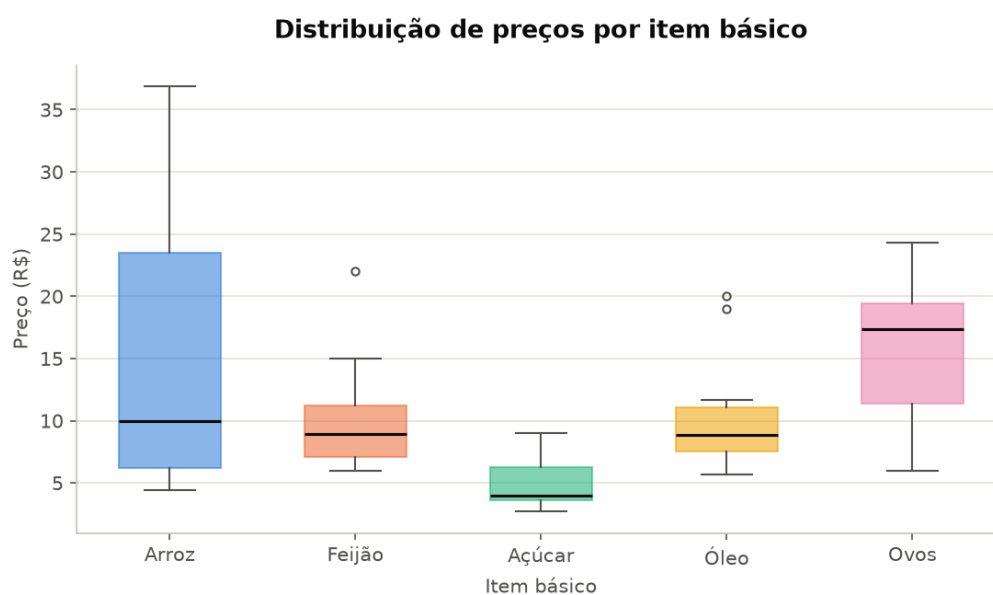


Figura 2 — Distribuição (boxplot) de preços por item básico.

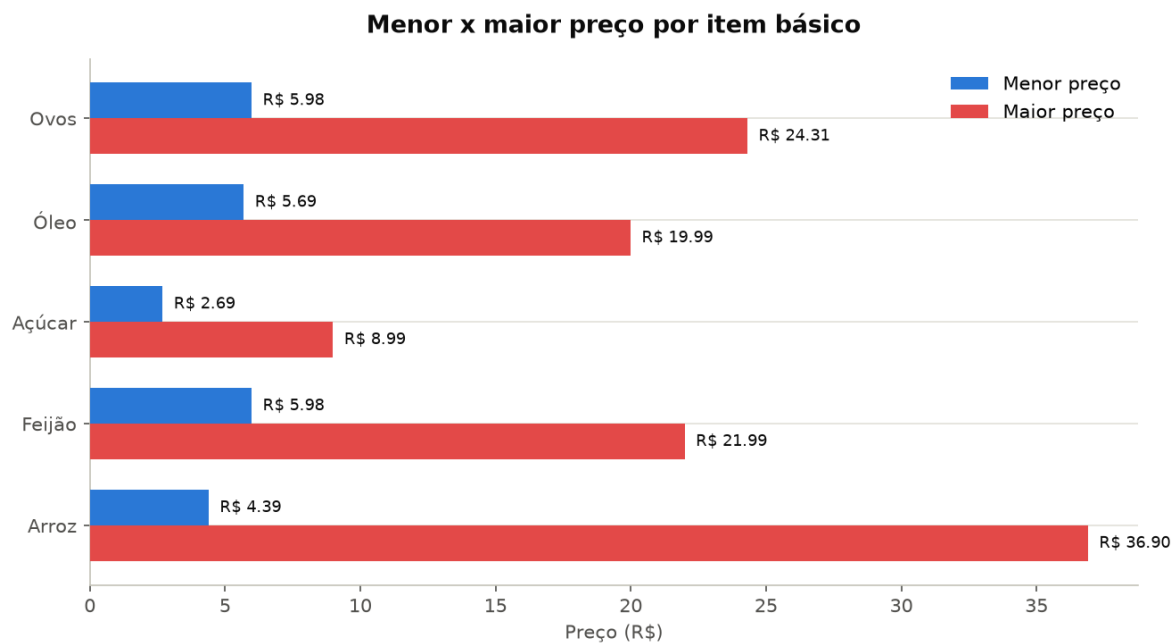


Figura 3 — Produto mais barato x mais caro por item básico.

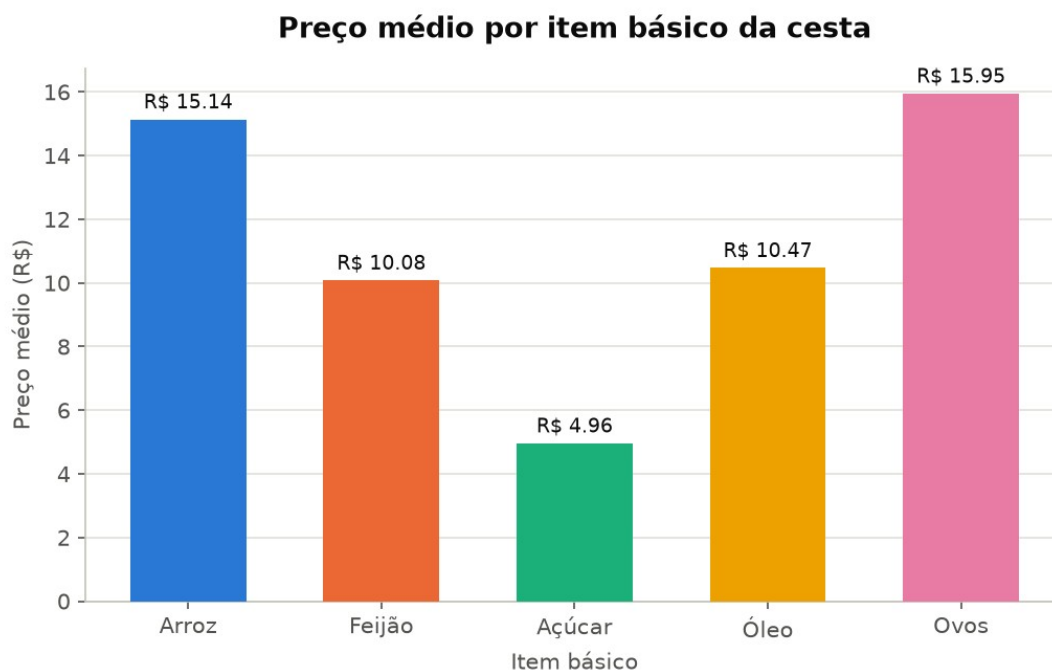


Figura 3 — Produto mais barato x mais caro por item básico.

5.4 Padrões identificados

1. **Ovos é o item mais caro em média (R\$ 15,95); açúcar o mais barato (R\$ 4,96).**
2. **Arroz tem a maior variação entre marcas (740%):** do mais barato (Arroz Branco Camil Tipo 1 1kg, R\$ 4,39, Carrefour) ao mais caro (ARROZ ARBÓRIO LA PASTINA 1KG, R\$ 36,90, São Luiz). Ressalva: são tipos distintos (branco comum vs. arbóreo italiano), então a diferença reflete variedade, não só marca.
3. **Açúcar tem a menor variação (234%),** sugerindo mercado mais padronizado.
4. **Arroz também tem a maior dispersão (desvio R\$ 10,15),** coerente com mais marcas e tipos.
5. **Produto mais barato de cada tipo, considerando as três fontes — todos no Carrefour:**

Item	Produto mais barato	Preço	Fonte
Arroz	Arroz Branco Camil Tipo 1 1kg	R\$ 4,39	Carrefour
Feijão	Feijão Preto Carrefour 1 Kg	R\$ 5,98	Carrefour
Açúcar	Açúcar Refinado Carrefour 1kg	R\$ 2,69	Carrefour
Óleo	Óleo de Soja Confiare 900ml	R\$ 5,69	Carrefour
Ovos	Ovos Brancos Jumbo Graciana Estojo 10un	R\$ 5,98	Carrefour

O CSV dados/mais_baratos_por_tipo.csv também inclui URL e código do produto.

7. Pipeline e Dashboard

Dashboard (dashboard.png, Python + Matplotlib, critério 5): combina em uma única figura o gráfico de preço médio por item e o gráfico de menor x maior preço, dando uma visão consolidada da cesta em um único painel.

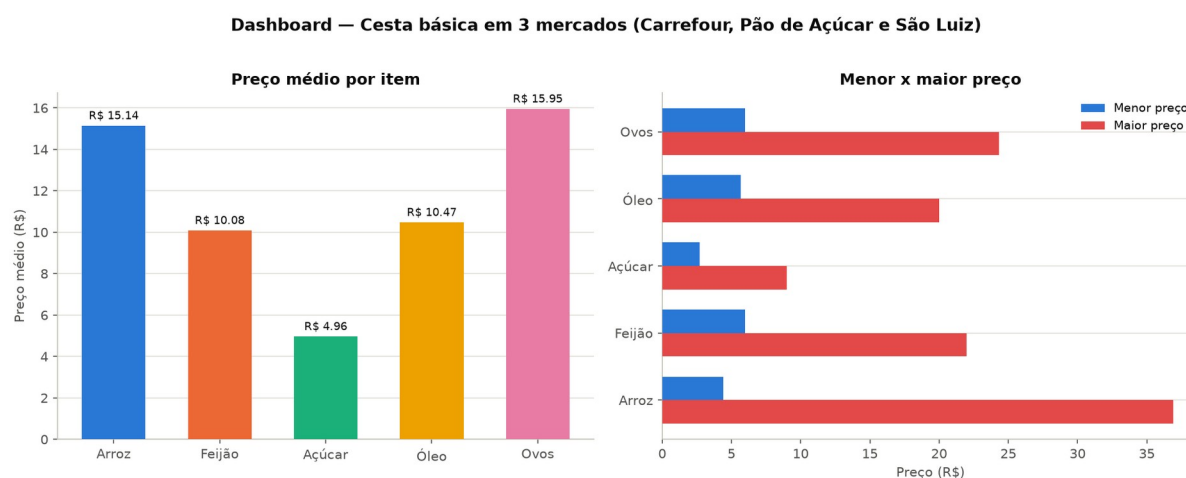
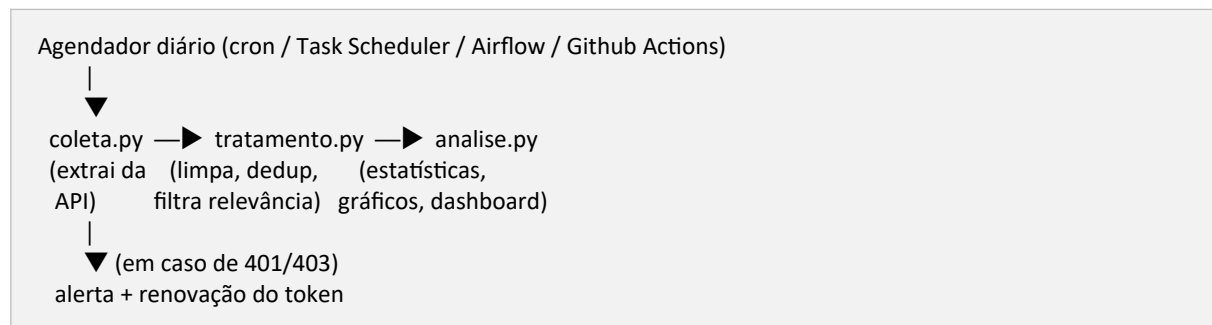


Figura 5 — Dashboard consolidado (preço médio + menor x maior preço).

Resposta à pergunta de negócio ("Quais marcas de cada item básico têm o menor e o maior preço no São Luiz, e qual a diferença entre elas?"), com base em `resumo_menor_maior_preco.csv` o `carrefour` ganhou em todos.

Pipeline de automação proposto (agendamento diário): um agendador (`cron`, `Task Scheduler` ou um serviço como `Airflow` ou `Github Actions`) dispararia `setup.py` diariamente, que faria as raspagens, depois o `tratamento.py` e `analise.py`. Os dados brutos são deletados e renovados junto com o token em cada execução.



8. Aspectos Éticos e Legais

Os dados coletados eram publicamente acessíveis? Sim. As informações de produto e preço estão disponíveis para qualquer visitante do site, sem necessidade de login ou cadastro.

Existia uma API que poderia ser utilizada como alternativa? A própria fonte de dados é consumida via API e seletores — porém não uma API pública e documentada, oferecida oficialmente para desenvolvedores externos, e sim uma interface interna do site, obtida por engenharia reversa do tráfego de rede (ver seção Método de Coleta). Não foi identificada, no escopo desta atividade, uma API pública oficial e documentada para os mesmos dados.

Foram coletados dados pessoais? Não. Os registros coletados contêm apenas informações de produtos (nome, preço, código de barras, estoque, categoria); nenhum dado de clientes, pedidos ou informações pessoais foi acessado ou armazenado.

A coleta poderia causar sobrecarga ao servidor? Sim, caso feita sem controle — e isso foi observado na prática: uma sequência de requisições em ritmo elevado (mesmo com pausa de 1 segundo entre chamadas) foi suficiente para que um mecanismo de proteção do servidor retornasse erro 403. Isso confirma que o serviço monitora e limita o volume de requisições por origem.

Que cuidados deveriam ser adotados caso esse processo fosse utilizado continuamente por uma empresa? Com base na experiência prática desta atividade, recomenda-se:

1. Limitar a frequência e o volume de requisições (rate limiting no lado do cliente, com pausas e backoff progressivo em caso de erro).

2. Coletar apenas os dados estritamente necessários para a análise, em vez de varrer o catálogo completo.
3. Tratar o token de autenticação como uma credencial sensível (nunca versionado em repositórios públicos, armazenado via variável de ambiente).
4. Monitorar respostas de erro (401/403/429) para identificar bloqueios e ajustar o comportamento automaticamente.
5. Avaliar, junto à empresa proprietária do site, a possibilidade de acordo formal de uso de dados, já que se trata de uma API não documentada publicamente.

Três boas práticas adotadas nesta coleta:

1. Pausa entre requisições consecutivas, para não sobrecarregar o servidor.
2. Limite de 5 itens por categoria, coletando apenas o volume necessário para a análise (não uma extração massiva do catálogo completo).
3. Token de autenticação mantido fora do código-fonte (variável de ambiente), evitando exposição de credenciais em caso de publicação do código.